FSDH PAYHUB MERCHANT

Codebase Guide

A plain-English walk through the project on the `akpan-wip` branch: what every file does, how the files talk to each other, and a step-by-step plan for building the Sign In and Overview screens from the Figma designs.

Prepared for Otani · 22 October 2026
Project folder: `C:\Users\makpan\payhubmerchant` · Branch: `akpan-wip`

What is inside this guide

1. The project in one page
2. How the app starts up (the journey from browser to screen)
3. Every file explained
4. How the files connect to each other
5. Things to tidy up before you build
6. Packages: what is installed and what you need to add
7. Building the Sign In screen, step by step
8. Building the Overview screen, step by step
9. Your folder structure when you are done
10. Order of work and a final checklist
11. Small dictionary of words used in this guide

1. The project in one page

This is a **website that runs in the browser**, written in **React** (a library for building screens out of small reusable pieces called *components*) and **TypeScript** (JavaScript with types, so mistakes are caught early). The files end in `.tsx` when they contain screen code and `.ts` when they contain plain logic.

The tools around it:

- **Vite** runs the app on your computer while you work and builds it for release.
- **Material UI (MUI)** gives ready-made pieces like buttons, text boxes, tables and layout boxes.
- **Tailwind CSS** is also set up (for styling with short class names) but almost nothing uses it yet. The existing screens use MUI's `sx` styling instead.
- **Redux Toolkit** is the app's shared memory. It remembers who is logged in so any screen can ask.
- **React Router** decides which screen to show for each web address (for example `/dashboard`).

> **Important background**
> This project was copied from an older FSDH *staff onboarding* app. You will see leftovers from that app everywhere: the browser tab title says "HR Process Automation", users are grouped as "fulltime", "interns" and "consultant", and the API talks about "employees" and "onboarding". None of this is wrong for running the app, but it is not about Payhub. Section 5 lists what to clean up.

How to run it

1. Open a terminal in the project folder.

2. Run `npm install` once (downloads the packages into `node_modules`).
3. Run `npm run dev`.
4. Open `http://localhost:9090` in your browser.

The project expects Node version 22 (it says so in `package.json` under "volta").

2. How the app starts up

When someone opens the site, the files run in this order:

index.html → src/index.tsx → src/App.tsx → AuthBootstrap → Routes (pick a page) → The page on screen

1. **index.html** is the only real HTML page. It has an empty box called `root` and loads `src/index.tsx`.
2. **src/index.tsx** loads the global styles (`index.css`) and puts the `App` component inside that empty box.
3. **src/App.tsx** wraps everything in three "providers" (think of them as power sockets the whole app can plug into): the Redux store (shared memory), the MUI theme (colours and fonts), and the router (web addresses).
4. **AuthBootstrap** runs once at start. It looks in the browser's saved storage (`localStorage`) to see if someone was already logged in, and if so puts them back into the shared memory.
5. **Routes** look at the address bar and choose a page. For example `/` shows the landing page and `/dashboard` shows the dashboard.

The pages (routes) that exist today

Address	Page shown	Who can see it
`/`	LandingPage. Has a "LOGIN FORM HERE" placeholder. **This is where your Sign In screen goes.**	Everyone
`/session/:token`	SessionLanding. Checks a "magic link" token with the server, then logs the user in.	Everyone
`/dashboard`	Dashboard. Currently an empty page with a sidebar. **This is where your Overview screen goes.**	Logged in, type "fulltime"
`/dboard`	Same Dashboard	Logged in, type "interns"
`/consultant-dashboard`	Same Dashboard	Logged in, type "consultant"
`*` (anything else)	NotFound (the 404 page)	Everyone

3. Every file explained

Files in the main folder (settings files)

You will rarely need to change these. They tell the tools how to behave.

File	What it does
`package.json`	The shopping list. Names every package the app uses and the short commands you can run (`npm run dev` , `npm run build` , `npm run lint`).
`package-lock.json`	The exact versions that were installed, so everyone gets the same thing. Never edit by hand.
`index.html`	The single HTML page. Sets the browser tab title and icon and loads the app.
`vite.config.ts`	Settings for Vite (the dev server and builder). Only switches on React support.
`tsconfig.json`	Settings for TypeScript. "strict" is on, so unused variables and type mistakes show as errors.
`tsconfig.node.json`	TypeScript settings just for `vite.config.ts` .
`tailwind.config.js`	Tailwind settings. Adds two colour names, `primary` and `light` , that read from CSS variables in `index.css` .
`postcss.config.js`	Plugs Tailwind and Autoprefixer into the CSS build. Leave alone.
`.eslintrc.cjs`	Rules for the code checker (`npm run lint`), which warns about bad patterns.
`.gitignore`	Files Git should not track, such as `node_modules` and `.env` .
`.env`	Your local settings: the API server address (`VITE_API_BASE_URL`) and two testing switches, `VITE_USE_MOCK_TOKEN` and `VITE_MOCK_EMPLOYEE_TYPE` .
`.env.production`	The same settings, used when building for release.
`README.md`	Short start-up notes and what the two testing switches do.
`public/`	Files served as-is: `favicon.png` (tab icon), `fsdhlogo.png` and `FSDH 1.png` (logos). Use them in code as `/fsdhlogo.png` .

src/ (the app itself)

File	What it does

`index.tsx`	The starting point. Loads `index.css` and draws `<App />` on the page.
`index.css`	Global styles. Loads the fonts (Merriweather for headings, Avenir for body), switches on Tailwind, and sets the CSS colour `--primary: #004987` (FSDH blue).
`App.tsx`	The heart of the app. Creates the MUI theme (FSDH blue, fonts), wraps everything in the store, theme and router, lists all the routes, and shows the "Session Expired" pop-up when the server says the login has run out.
`vite-env.d.ts`	Tells TypeScript which `VITE_...` settings exist in `.env`, so `import.meta.env.VITE_API_BASE_URL` is type-checked.

src/components/ (reusable building blocks)

File	What it does
`AuthBootstrap.tsx`	Invisible helper that runs on start. Reads the saved user and token from `localStorage` and puts them back into Redux and the API client. Without it you would be logged out on every page refresh.
`ProtectedRoute.tsx`	A gatekeeper. Wrap a page in it and only logged-in users of the allowed type get in; everyone else is sent back to `/`.
`DashboardLayout.tsx`	The frame for logged-in pages: the sidebar on the left (280px wide) and the page content on the right.
`Sidebar.tsx`	The left menu. Shows the FSDH name, the menu links it is given, and the user's initials, name, email and a logout button at the bottom. Logout clears storage, Redux and the token, then goes to `/`.

src/config/ (fixed settings)

File	What it does
`navigation.ts`	The list of sidebar menu items for each user type. Today each list has only "Dashboard".
`sessionKeys.ts`	The names used when saving things in `localStorage` (user, token, employee id, onboarding data). Kept in one place so every file uses the same names.

src/pages/ (full screens)

File	What it does

`auth/LandingPage.tsx`	The page at `/`. A two-column layout: left card with the FSDH logo, a "PAY HUB PORTAL" badge and the text "LOGIN FORM HERE" (with a `{/* //START HERE */}` comment); right card with the `smiling-lady.png` picture. This is the placeholder for your Sign In screen.
`auth/SessionLanding.tsx`	The page at `/session/:token`. Takes the token from the link, asks the server if it is valid, builds the user, saves everything, and sends the user to the right dashboard. In mock mode it skips the server and logs in a fake user "John Okafor". This is the best example in the project of a complete login flow, so copy its pattern.
`auth/AdminSignIn.tsx`, `EmployeeSignIn.tsx`, `InternSignIn.tsx`	Empty leftovers. Each one just redirects to `/`. Nothing uses them. Safe to delete later.
`Dashboard.tsx`	Picks the right menu list for the user type and shows `DashboardLayout` with no content inside. Your Overview content goes here (or in a new page that replaces it).
`NotFound.tsx`	The friendly "404 Page not found" screen with a Go Back button.

src/services/ (talking to the server)

File	What it does
`api/httpClient.ts`	The one place that actually sends requests to the server. It adds the server address from `.env`, attaches the login token (`Authorization: Bearer ...`), reads the reply, turns errors into readable messages, and, if the server answers 401 (not allowed), logs the user out and fires the "session expired" event that `App.tsx` listens for. Gives you `apiClient.get / post / put / patch / delete`.
`api/index.ts`	A shortcut file that re-exports the useful parts of `httpClient.ts` so other files can import from `services/api`.
`api/employeesService.ts`	A list of ready-made server calls for the old onboarding app (get employees, verify magic link, and so on). Each one is a one-line wrapper around `apiClient`. Use it as a pattern for your own `authService.ts` and `overviewService.ts`.
`onboardingPrefill.ts`	A large (25 KB) helper from the onboarding app that digs through server replies to pre-fill onboarding forms. Only `SessionLanding.tsx` uses it. You do not need it for Payhub.

src/store/ (shared memory, Redux)

File	What it does

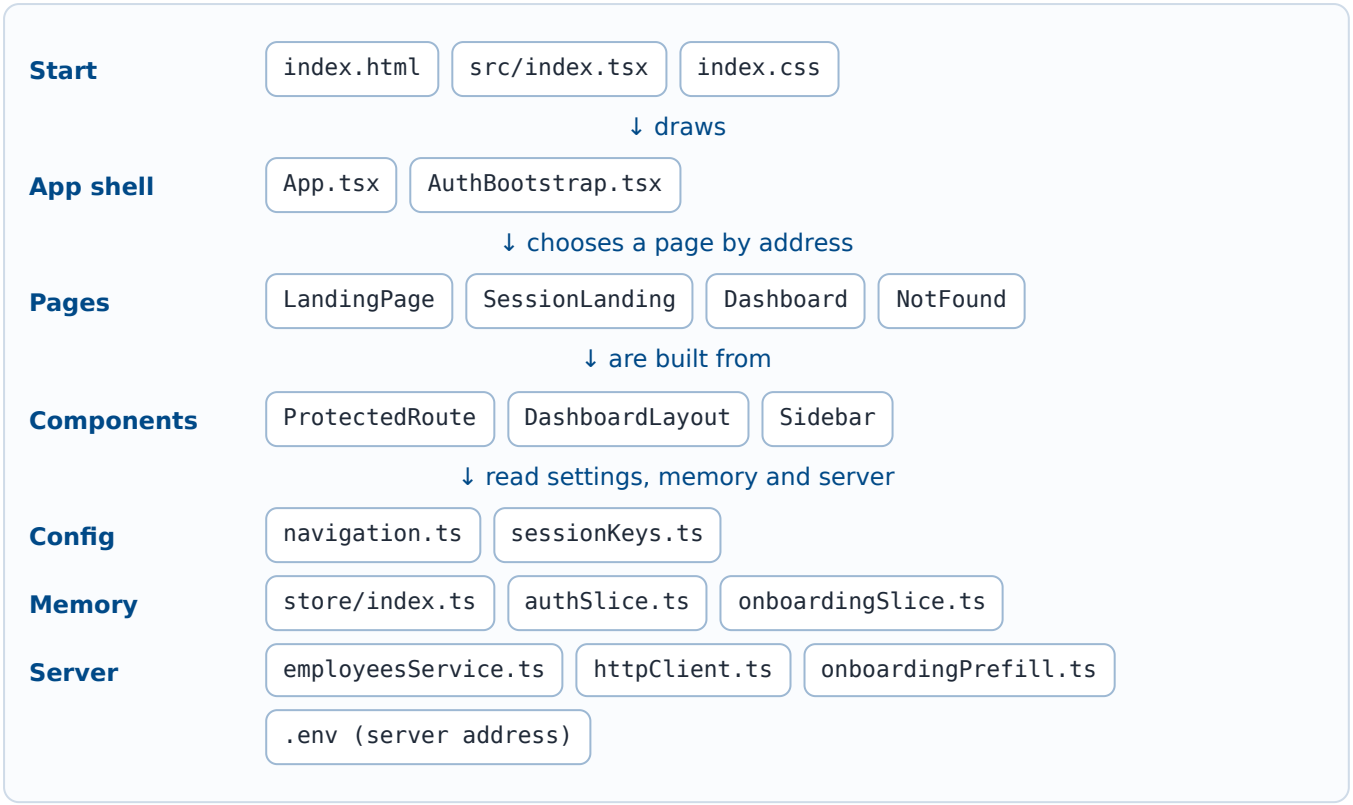
`index.ts`	Creates the store with two sections ("slices"): `auth` and `onboarding`. Also gives you `useAppSelector` (read from memory) and `useAppDispatch` (send an action to change memory). Always use these two instead of the plain Redux ones.
`authSlice.ts`	Remembers the logged-in `user`, `isAuthenticated` and `isHydrated` ("have we finished checking storage yet"). Actions: `setAuthenticatedUser`, `logout`, `setAuthHydrated`. Also defines the `User` shape and the three user types.
`onboardingSlice.ts`	Remembers onboarding pre-fill data. Cleared on logout. Not needed for Payhub.

src/assets/images/

File	What it is
`First SIgn.png`	The Figma design of the Sign In screen (a picture, not code).
`Overview Screen.png`	The Figma design of the Overview dashboard (a picture, not code).
`smiling-lady.png`	Photo used on the current placeholder landing page. The new design does not use it.

4. How the files connect to each other

Read this from top to bottom. Each row uses things from the rows below it.



Who imports whom

File	Uses these files
`index.tsx`	App.tsx, index.css
`App.tsx`	store, httpClient (session-expired event), ProtectedRoute, AuthBootstrap, LandingPage, SessionLanding, Dashboard, NotFound
`AuthBootstrap.tsx`	services/api, store, authSlice, onboardingSlice, sessionKeys
`ProtectedRoute.tsx`	store, authSlice (UserType)
`Dashboard.tsx`	DashboardLayout, navigation.ts, store
`DashboardLayout.tsx`	Sidebar
`Sidebar.tsx`	store, authSlice, onboardingSlice, services/api, sessionKeys
`LandingPage.tsx`	smiling-lady.png
`SessionLanding.tsx`	services/api, employeesService, store, authSlice, onboardingSlice, sessionKeys, onboardingPrefill
`employeesService.ts`	httpClient

`httpClient.ts`	store, authSlice, sessionKeys, .env
`store/index.ts`	authSlice, onboardingSlice

Three stories that show the connections in action

Story 1: Logging in (today, with a magic link)

1. The user clicks an emailed link like `/session/abc123`.
2. `App.tsx` sees the address and shows `SessionLanding`.
3. `SessionLanding` calls `employeesService.verifyMagicLink("abc123")`, which calls `apiClient.post` in `httpClient.ts`, which sends the request to the server address in `.env`.
4. The server replies with user details and a token. `SessionLanding` saves them in `localStorage` (using names from `sessionKeys.ts`), gives the token to `httpClient` with `setApiAccessToken`, and tells Redux with `setAuthenticatedUser`.
5. It then goes to `/dashboard`. `ProtectedRoute` checks Redux, sees a logged-in "fulltime" user, and lets them in.

Story 2: Refreshing the page

1. Refreshing wipes Redux memory. `ProtectedRoute` waits (shows nothing) while `isHydrated` is false.
2. `AuthBootstrap` reads the saved user and token from `localStorage` and puts them back. Now `isHydrated` is true and the page appears.

Story 3: The login runs out

1. Any server call that gets a 401 reply goes through `httpClient.ts`.
2. It clears storage, dispatches `logout` to Redux, and fires the `employee:session-expired` event.
3. `App.tsx` hears the event and shows "Session Expired! Kindly login Again." `ProtectedRoute` now sees no user and sends them to `/`.

> **Testing without a server**
>
> In `.env` set `VITE_USE_MOCK_TOKEN=true` and `VITE_MOCK_EMPLOYEE_TYPE=fulltime`, restart `npm run dev`, then open `http://localhost:9090/session/anything`. You will be logged in as a fake user and land on `/dashboard`. This is the quickest way to see your Overview screen while you build it.

5. Things to tidy up before you build

These are small problems I found while reading the code. None of them stop the app from running today, but fixing them first will save you confusion.

Where	Problem	Fix
`src/index.tsx`	Uses the old `render` function. React 18 shows a warning in the console and runs in "old mode".	Replace with the code shown below.
`src/index.css`	`--light: #;` is not a real colour.	Give it a value, for example `--light: #F3F9FF;` (the Overview page background).
`src/index.css`	Avenir is loaded from Google Fonts, but Google Fonts does not have Avenir, so that line silently does nothing. The browser falls back to a default font.	See "Fonts" in section 6.
`index.html`	Tab title says "HR Process Automation | Seamless Staff Onboarding Experience".	Change to `FSDH Payhub`.
`package.json`	`react-redux`, `autoprefixer` and `postcss` are set to "latest", so a fresh install might pull a newer version that breaks.	Pin them to the versions in `package-lock.json` (for example `"react-redux": "^9.2.0"`; check the lock file for the exact numbers).
`App.tsx`	The `/testing` route is listed after `*`. It works, but it is a leftover.	Remove it.
`pages/auth/*SignIn.tsx`	Three unused redirect files.	Delete when you add your real `SignIn.tsx`.

New `src/index.tsx`:

```
import "./index.css";
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { App } from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

About user types

`ProtectedRoute` only lets in users whose type is "fulltime", "interns" or "consultant". Payhub users are merchants, not staff. For now the simplest path is: when your Sign In succeeds, set `userType: 'fulltime'` so `/dashboard` lets them in. Later, when you know the real Payhub roles from the backend team (for example "initiator", "approver", "admin"), change the `UserType` list in `authSlice.ts` and the routes in `App.tsx` together.

6. Packages: what you have and what to add

Already installed (you do not need to install these)

Package	What you will use it for
`@mui/material` + `@emotion/react` + `@emotion/styled`	All the building blocks: `Box`, `Typography`, `TextField`, `Button`, `Table`, `Tabs`, `Chip`, `Avatar`, `IconButton`, `InputAdornment`. Emotion is the styling engine MUI needs.
`formik`	Handles the Sign In form: remembers what was typed, shows errors, runs submit.
`yup`	Rules for the form, such as "email is required and must look like an email".
`lucide-react`	All the icons in both designs (sidebar icons, bell, search, filter, check, warning, arrows).
`framer-motion`	Smooth slide animation for the carousel on the right of the Sign In screen.
`react-router-dom`	Moving between pages (`useNavigate`, `Route`).
`@reduxjs/toolkit` + `react-redux`	Saving the logged-in user after Sign In.
`tailwindcss`	Optional. You can style with Tailwind classes if you prefer, but stay consistent with the existing MUI `sx` style.
`react-pdf`	Not used anywhere yet. Not needed for these two screens.

Packages to install

Package	Why	Command
`recharts`	Draws the "Volume Trends" area chart and the "Inflow Channel" doughnut chart. Simple, well documented, works well with React.	`npm install recharts`

`@fontsource/nunito-sans` (only if you cannot get Avenir)	A free font that looks close to Avenir, bundled with the app so it always loads.	`npm install @fontsource/nunito-sans`

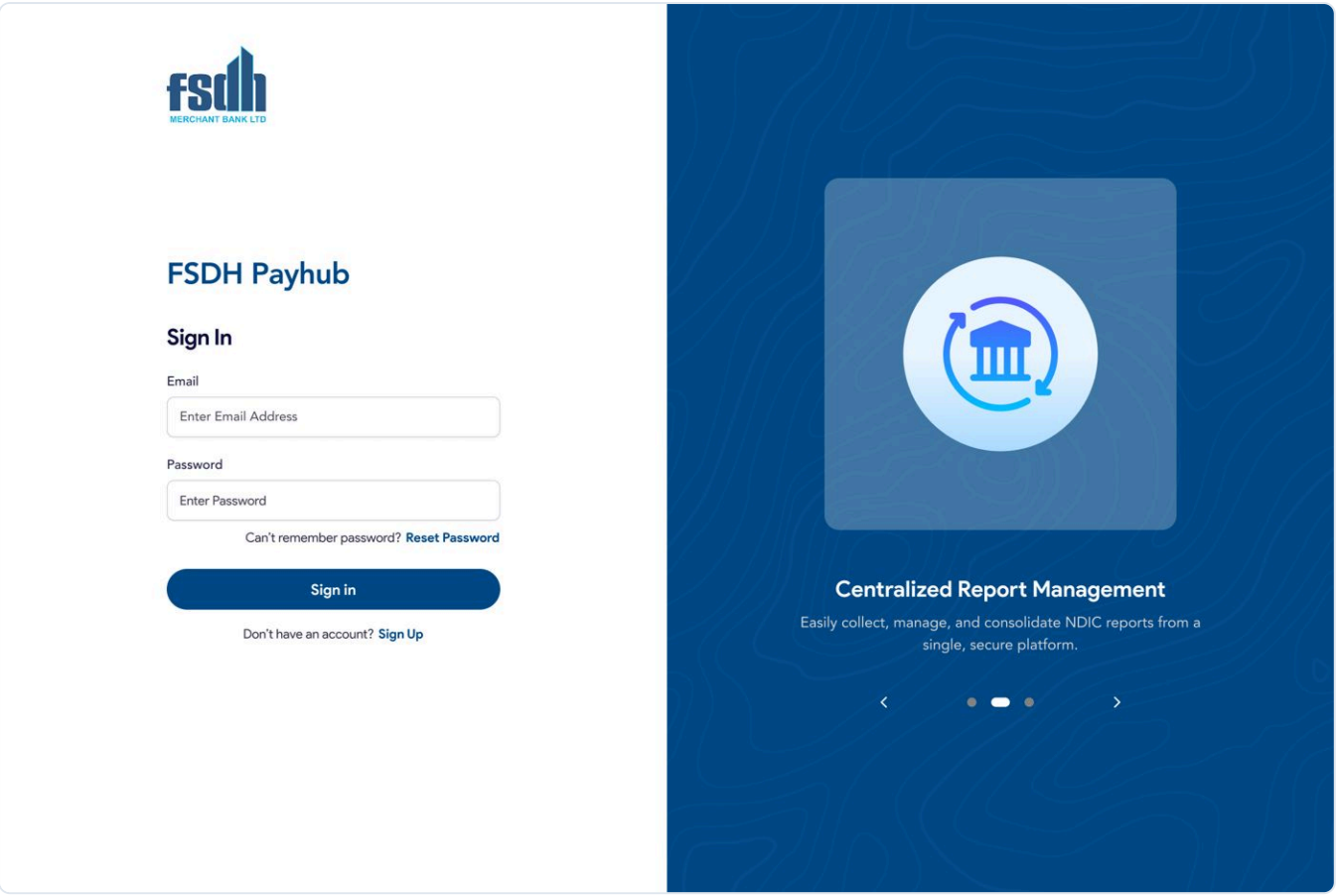
That is all. Everything else in the designs (cards, table, tabs, search box, pagination buttons, status pills, carousel) can be built with what you already have.

> **Fonts**
> The designs use **Avenir** for most text and a serif (Merriweather) for "FSDH Payhub" in the sidebar. Avenir is a paid font. Ask the designer or FSDH brand team for the font files (`.woff2`), put them in `src/assets/fonts/` , and load them with `@font-face` in `index.css` . If you cannot get them, install Nunito Sans (above), add `import '@fontsource/nunito-sans/400.css'` (and 600, 700) to `index.tsx` , and set `fontFamily: 'Avenir, "Nunito Sans", sans-serif'` in the theme in `App.tsx` . Delete the broken Avenir Google Fonts line either way.

Optional packages (only if you want them later)

- `@mui/x-data-grid` : a powerful table with built-in sorting and paging. Not needed now; a plain MUI `Table` matches the design better.
- `dayjs` : easier date formatting (for "Friday, 23rd July 2026"). You can also use plain `toLocaleDateString` .

7. Building the Sign In screen, step by step



What is on the screen

Left half (white)

- FSDH Merchant Bank logo, top left
- "FSDH Payhub" heading in FSDH blue
- "Sign In" sub-heading in dark navy
- Email field with label
- Password field with label
- "Can't remember password? **Reset Password**" right-aligned
- Full-width rounded blue "Sign in" button
- "Don't have an account? **Sign Up**" centred

Right half (FSDH blue)

- Blue background with faint contour-line pattern
- A see-through lighter square holding a white circle and a bank icon
- Title "Centralized Report Management"
- Short description under it
- Left and right arrows, plus three dots (the middle one is a wider pill = current slide). So this is a **carousel** of 3 slides.

Colours and sizes (from the design)

Thing	Value

FSDH blue (heading, button, right panel)	`#004987`
"Sign In" text	very dark navy, about `#0B1C3D`
Card on right	white at about 25% see-through: `rgba(255,255,255,0.25)`, rounded 16px
Input boxes	white, light grey border `#D0D5DD`, rounded 8px, about 56px tall
Button	FSDH blue, fully rounded (999px), about 56px tall, white bold text
Form width	about 460px, starting about a quarter of the way in on the left half

These were measured from the picture, so they are close but not exact. Open the design in Figma and click an item to see its exact colour, size and font in the right-hand "Inspect" panel.

Step 1. Get the pictures out of Figma

In Figma, select each item, then in the right panel under **Export** choose the format and click Export:

- FSDH Merchant Bank logo → SVG → save as `src/assets/images/fsdh-merchant-logo.svg`
- The blue contour background → PNG at 2x or SVG → `src/assets/images/auth-bg-pattern.png`
- Each slide's circle illustration (3 of them) → SVG or PNG → `src/assets/images/slide-1.svg`, `slide-2.svg`, `slide-3.svg`

Also ask the designer for the title and text of slides 1 and 3 (only slide 2 is visible in the image).

Step 2. Create the files

```
src/pages/auth/SignIn.tsx              ← the page (replaces LandingPage)
src/components/auth/AuthCarousel.tsx   ← the right-hand blue panel
src/services/api/authService.ts        ← the login server call
```

Step 3. Build the page layout

Two equal columns that fill the screen. On small screens hide the blue panel so the form gets the full width.

```tsx
// src/pages/auth/SignIn.tsx
import { Box } from '@mui/material';
import AuthCarousel from '../../components/auth/AuthCarousel';
import SignInForm from './SignInForm';   // or put the form right here

export default function SignIn() {
  return (
    <Box sx={{ minHeight: '100vh', display: 'grid',
               gridTemplateColumns: { xs: '1fr', md: '1fr 1fr' } }}>
      <Box sx={{ bgcolor: '#fff', px: { xs: 3, md: 12 }, py: 6 }}>
        <SignInForm />
      </Box>
      <Box sx={{ display: { xs: 'none', md: 'block' } }}>
        <AuthCarousel />
      </Box>
    </Box>
  );
}
```

Step 4. Build the form with Formik and Yup

```
// src/pages/auth/SignInForm.tsx
import { useState } from 'react';
import { Box, Typography, TextField, Button, Link, Alert,
         InputAdornment, IconButton } from '@mui/material';
import { Eye, EyeOff } from 'lucide-react';
import { useFormik } from 'formik';
import * as yup from 'yup';
import { useNavigate } from 'react-router-dom';
import logo from '../../assets/images/fsdh-merchant-logo.svg';
import { useAppDispatch } from '../../store';
import { signIn } from '../../services/auth/signIn';

const schema = yup.object({
  email: yup.string().email('Enter a valid email').required('Email is required'),
  password: yup.string().required('Password is required'),
});

export default function SignInForm() {
  const [showPassword, setShowPassword] = useState(false);
  const [serverError, setServerError] = useState('');
  const navigate = useNavigate();
  const dispatch = useAppDispatch();

  const formik = useFormik({
    initialValues: { email: '', password: '' },
    validationSchema: schema,
    onSubmit: async (values) => {
      setServerError('');
      try {
        await signIn(values.email, values.password, dispatch);
        navigate('/dashboard', { replace: true });
      } catch (err) {
        setServerError(err instanceof Error ? err.message : 'Sign in failed');
      }
    },
  });

  const inputSx = { '& .MuiOutlinedInput-root': { borderRadius: '8px', height: 56 } };

  return (
    <Box sx={{ maxWidth: 460 }}>
      <Box component="img" src={logo} alt="FSDH Merchant Bank" sx={{ width: 150, mb: 16 }} />
      <Typography sx={{ color: '#004987', fontSize: 40, fontWeight: 600, mb: 4 }}>
        FSDH Payhub
      </Typography>
      <Typography sx={{ color: '#0B1C3D', fontSize: 30, fontWeight: 600, mb: 3 }}>
        Sign In
      </Typography>

      {serverError && <Alert severity="error" sx={{ mb: 2 }}>{serverError}</Alert>}

      <form onSubmit={formik.handleSubmit} noValidate>
        <Typography component="label" htmlFor="email">Email</Typography>
        <TextField id="email" name="email" fullWidth placeholder="Enter Email Address"
          value={formik.values.email} onChange={formik.handleChange} onBlur={formik.handleBlur}
          error={formik.touched.email && Boolean(formik.errors.email)}
          helperText={formik.touched.email && formik.errors.email}
          sx={{ mt: 1, mb: 3, ...inputSx }} />
```

```
        <Typography component="label" htmlFor="password">Password</Typography>
        <TextField id="password" name="password" fullWidth placeholder="Enter Password"
          type={showPassword ? 'text' : 'password'}
          value={formik.values.password} onChange={formik.handleChange} onBlur=
{formik.handleBlur}
          error={formik.touched.password && Boolean(formik.errors.password)}
          helperText={formik.touched.password && formik.errors.password}
          sx={{ mt: 1, ...inputSx }}
          InputProps={{ endAdornment: (
            <InputAdornment position="end">
              <IconButton onClick={() => setShowPassword((s) => !s)} edge="end">
                {showPassword ? <EyeOff size={18} /> : <Eye size={18} />}
              </IconButton>
            </InputAdornment>) }} />

        <Typography sx={{ textAlign: 'right', mt: 1.5, mb: 4, fontSize: 15 }}>
          Can't remember password?{' '}
          <Link href="#" underline="none" sx={{ fontWeight: 700 }}>Reset Password</Link>
        </Typography>

        <Button type="submit" fullWidth variant="contained" disabled={formik.isSubmitting}
          sx={{ height: 56, borderRadius: 999, textTransform: 'none',
                fontSize: 17, fontWeight: 600, boxShadow: 'none' }}>
          {formik.isSubmitting ? 'Signing in...' : 'Sign in'}
        </Button>
      </form>

      <Typography sx={{ textAlign: 'center', mt: 3, fontSize: 15 }}>
        Don't have an account?{' '}
        <Link href="#" underline="none" sx={{ fontWeight: 700 }}>Sign Up</Link>
      </Typography>
    </Box>
  );
}
```

The eye icon to show or hide the password is not in the design but is a common, helpful extra. Leave it out if the designer prefers. Importing an `.svg` like this works out of the box in Vite.

Step 5. Build the carousel

Keep the slides in a list. Clicking an arrow or a dot changes which slide is showing. `framer-motion` fades and slides between them.

```
// src/components/auth/AuthCarousel.tsx
import { useEffect, useState } from 'react';
import { Box, Typography, IconButton } from '@mui/material';
import { ChevronLeft, ChevronRight } from 'lucide-react';
import { AnimatePresence, motion } from 'framer-motion';
import bg from '../../assets/images/auth-bg-pattern.png';
import slide1 from '../../assets/images/slide-1.svg';
import slide2 from '../../assets/images/slide-2.svg';
import slide3 from '../../assets/images/slide-3.svg';

const slides = [
  { image: slide1, title: 'Slide 1 title from Figma', text: 'Slide 1 text from Figma' },
  { image: slide2, title: 'Centralized Report Management',
    text: 'Easily collect, manage, and consolidate NDIC reports from a single, secure platform.'
},
  { image: slide3, title: 'Slide 3 title from Figma', text: 'Slide 3 text from Figma' },
];

export default function AuthCarousel() {
  const [index, setIndex] = useState(1);
  const go = (step: number) => setIndex((i) => (i + step + slides.length) % slides.length);

  useEffect(() => {                       // move on by itself every 6 seconds
    const t = setInterval(() => go(1), 6000);
    return () => clearInterval(t);
  }, []);

  const slide = slides[index];
  return (
    <Box sx={{ height: '100%', minHeight: '100vh', bgcolor: '#004987',
               backgroundImage: `url(${bg})`, backgroundSize: 'cover',
               display: 'flex', flexDirection: 'column', alignItems: 'center',
               justifyContent: 'center', color: '#fff', px: 6 }}>
      <AnimatePresence mode="wait">
        <motion.div key={index} initial={{ opacity: 0, x: 40 }}
          animate={{ opacity: 1, x: 0 }} exit={{ opacity: 0, x: -40 }}
          transition={{ duration: 0.35 }} style={{ textAlign: 'center' }}>
          <Box sx={{ width: 360, height: 360, mx: 'auto', borderRadius: '16px',
                     bgcolor: 'rgba(255,255,255,0.25)', display: 'grid', placeItems: 'center'
}}>
            <Box component="img" src={slide.image} alt="" sx={{ width: 200 }} />
          </Box>
          <Typography sx={{ fontSize: 26, fontWeight: 600, mt: 6 }}>{slide.title}</Typography>
          <Typography sx={{ maxWidth: 420, mx: 'auto', mt: 1, opacity: 0.9 }}>{slide.text}
</Typography>
        </motion.div>
      </AnimatePresence>

      <Box sx={{ display: 'flex', alignItems: 'center', gap: 10, mt: 5 }}>
        <IconButton onClick={() => go(-1)} sx={{ color: '#fff' }}><ChevronLeft /></IconButton>
        <Box sx={{ display: 'flex', gap: 1.5 }}>
          {slides.map((_, i) => (
            <Box key={i} onClick={() => setIndex(i)} sx={{ cursor: 'pointer',
              height: 10, width: i === index ? 28 : 10, borderRadius: 999,
              bgcolor: i === index ? '#fff' : 'rgba(255,255,255,0.45)',
              transition: 'width .3s' }} />
          ))}
        </Box>
        <IconButton onClick={() => go(1)} sx={{ color: '#fff' }}><ChevronRight /></IconButton>
      </Box>
```

```
    </Box>
  );
}
```

Step 6. Connect the form to the server

The project does not have an email-and-password login call yet (today it only has the magic link). Ask the backend team for the login address and what it sends back. Then:

6a. Add the server call, following the same pattern as `employeesService.ts`:

```
// src/services/api/authService.ts
import { apiClient } from './httpClient';

export interface LoginResponse {
  token?: string;
  accessToken?: string;
  user?: { id?: string; name?: string; email?: string; role?: string };
}

export const authService = {
  login(email: string, password: string) {
    return apiClient.post<LoginResponse>('/api/v1/auth/login', { email, password }); // confirm
path with backend
  },
};
```

6b. Add a small helper that logs in and saves the session. It copies what `SessionLanding.tsx` already does, so refresh and logout keep working:

```
// src/services/auth/signIn.ts
import { authService } from '../api/authService';
import { setApiAccessToken } from '../api';
import { setAuthenticatedUser, type User } from '../../store/authSlice';
import { SESSION_TOKEN_KEY, SESSION_USER_KEY } from '../../config/sessionKeys';
import type { AppDispatch } from '../../store';

export async function signIn(email: string, password: string, dispatch: AppDispatch) {
  const res = await authService.login(email, password);
  const token = res.token ?? res.accessToken ?? '';
  const name = res.user?.name ?? email;
  const user: User = {
    employeeId: res.user?.id ?? '',
    name,
    email: res.user?.email ?? email,
    role: res.user?.role ?? 'merchant',
    initials: name.split(' ').slice(0, 2).map((p) => p[0]?.toUpperCase()).join(''),
    userType: 'fulltime',                // temporary: see "About user types" in section 5
  };
  window.localStorage.setItem(SESSION_USER_KEY, JSON.stringify(user));
  window.localStorage.setItem(SESSION_TOKEN_KEY, token);
  setApiAccessToken(token);
  dispatch(setAuthenticatedUser(user));
}
```

No backend yet?
Until the login address exists, make `authService.login` wait half a second and return `{ token: 'mock', user: { name: 'Jamiu Akintunde', email } }` when `import.meta.env.VITE_USE_MOCK_TOKEN === 'true'`. You can then click "Sign in" with any email and land on the dashboard.

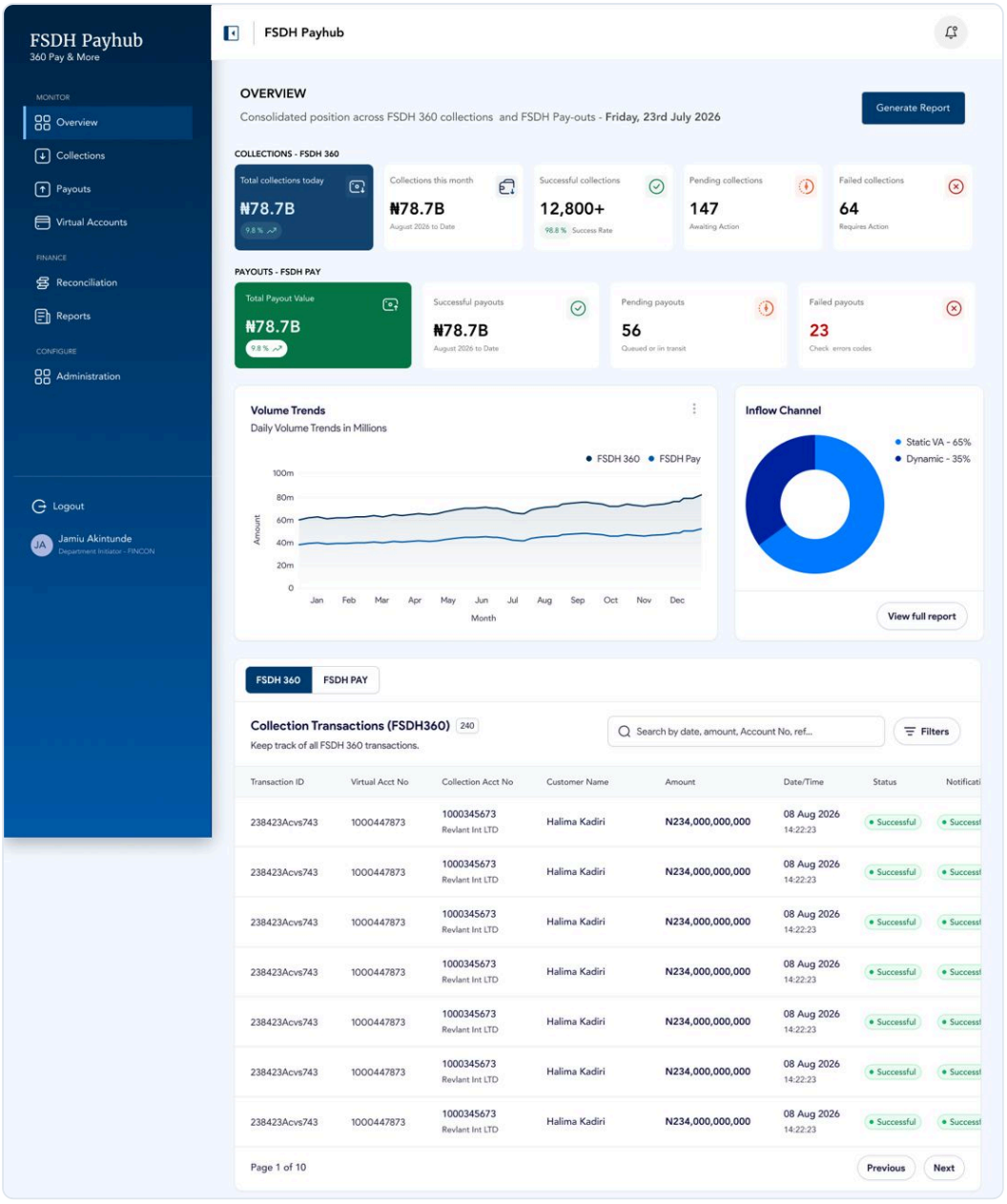
Step 7. Show the new page at `/`

In `App.tsx`, change the import and the first route:

```
import SignIn from './pages/auth/SignIn';
...
<Route path="/" element={<SignIn />} />
```

Keep `LandingPage.tsx` until the new page works, then delete it and `smiling-lady.png`.

8. Building the Overview screen, step by step



Break the screen into pieces first

Never build a big screen in one file. Look at the design and name each piece. Pieces that repeat become one component used many times.

Piece	Component name	Notes
Dark blue left menu	`Sidebar` (exists, restyle it)	Grouped into MONITOR, FINANCE, CONFIGURE. Logout and user at the bottom.
White top bar	`TopBar` (new)	Collapse-menu icon, "FSDH Payhub", bell icon on the right.

"OVERVIEW" title, date line, Generate Report button	`PageHeader` (new)	Today's date is worked out in code.
The 9 number cards	`StatCard` (new, used 9 times)	3 looks: navy "highlight", green "highlight", and white "normal".
"COLLECTIONS - FSDH 360" / "PAYOUTS - FSDH PAY" labels	`SectionLabel` (new, tiny)	Small bold capital text.
Volume Trends	`VolumeTrendsChart` (new)	Area chart with two lines, Jan to Dec. Uses recharts.
Inflow Channel	`InflowChannelChart` (new)	Doughnut chart, 65% / 35%, legend, "View full report" button.
Transactions box	`TransactionsTable` (new)	FSDH 360 / FSDH PAY tabs, title with count, search box, Filters button, table, page numbers.
Green "Successful" badges	`StatusPill` (new)	Also needed for Pending and Failed.

Colours used in the Overview design

Use	Colour
Sidebar background (top to bottom gradient)	`#002544` → `#0159A6`
Active menu item, Generate Report button, active tab	`#003C70`
Page background	`#F3F9FF`
"Total collections today" card	about `#1D4E7E`
"Total Payout Value" card	`#067647`
Success (ticks, pills)	`#12B76A` on `#ECFDF3`
Pending (orange icons)	`#F79009`
Failed (red icons, "23")	`#D92D20`
Chart: FSDH 360 / Static VA	`#007AFF`
Chart: FSDH Pay / Dynamic	`#0023A2`
Card borders	`#EAECF0`
Grey small text	`#667085`

Measured from the picture. Confirm in Figma's Inspect panel. Put them in one file (Step 1) so you only change them in one place.

Step 1. Put the colours in one place

```
// src/config/colors.ts
export const colors = {
  primary: '#004987',
  navy: '#003C70',
  sidebarTop: '#002544',
  sidebarBottom: '#0159A6',
  pageBg: '#F3F9FF',
  border: '#EAECF0',
  textDark: '#101828',
  textGrey: '#667085',
  success: '#12B76A', successBg: '#ECFDF3',
  pending: '#F79009', pendingBg: '#FFFAEB',
  failed: '#D92D20', failedBg: '#FEF3F2',
  payoutGreen: '#067647',
  collectionNavy: '#1D4E7E',
  chartLight: '#007AFF',
  chartDark: '#0023A2',
};
```

Step 2. Install the chart package

```
npm install recharts
```

Step 3. Update the menu list (navigation.ts)

The design groups the menu into sections, so add a `section` to each item. Pages that do not exist yet can point to addresses you will build later.

```
// src/config/navigation.ts
import { LayoutGrid, ArrowDownToLine, ArrowUpFromLine, CreditCard,
         Layers, FileText, type LucideIcon } from 'lucide-react';

export interface NavItem { label: string; icon: LucideIcon; path: string; section: string; }

export const payhubNavItems: NavItem[] = [
  { section: 'MONITOR',   label: 'Overview',         icon: LayoutGrid,      path: '/dashboard'
},
  { section: 'MONITOR',   label: 'Collections',      icon: ArrowDownToLine, path: '/collections'
},
  { section: 'MONITOR',   label: 'Payouts',          icon: ArrowUpFromLine, path: '/payouts' },
  { section: 'MONITOR',   label: 'Virtual Accounts', icon: CreditCard,      path: '/virtual-
accounts' },
  { section: 'FINANCE',   label: 'Reconciliation',   icon: Layers,          path:
'/reconciliation' },
  { section: 'FINANCE',   label: 'Reports',          icon: FileText,        path: '/reports' },
  { section: 'CONFIGURE', label: 'Administration',   icon: LayoutGrid,      path:
'/administration' },
];
```

Then in `Dashboard.tsx` pass `payhubNavItems` instead of picking between the three old lists. Add `section: string` to `SidebarMenuItem` in `Sidebar.tsx` too.

Step 4. Restyle the Sidebar

Keep all the logic (logout, active item, user details). Change only the look:

- Background: `background: 'linear-gradient(180deg, #002544 0%, #0159A6 100%)'`, text white.
- Top: "FSDH Payhub" in Merriweather, about 24px, and "360 Pay & More" under it in small text.
- Group the items by `section`. Before each group, show the section name in small grey capitals (about 11px, 60% white).
- Active item: background `#003C70`, a 3px light-blue bar on its left edge, rounded 4px.
- Bottom: a thin divider line, "Logout" with the `LogOut` icon, then a round avatar with initials, name, and the role in small text ("Department Initiator - FINCON").

Grouping in code:

```
const sections = [...new Set(menuItems.map((m) => m.section))];
{sections.map((section) => (
  <Box key={section} sx={{ mb: 2 }}>
    <Typography sx={{ fontSize: 11, color: 'rgba(255,255,255,0.6)', px: 2, mb: 1 }}>
      {section}
    </Typography>
    {menuItems.filter((m) => m.section === section).map((item) => ( /* existing ListItemButton
*/ ))}
  </Box>
))}
```

Step 5. Add the TopBar and update DashboardLayout

```
// src/components/TopBar.tsx
import { Box, Typography, IconButton } from '@mui/material';
import { PanelLeft, Bell } from 'lucide-react';

export default function TopBar() {
  return (
    <Box sx={{ height: 72, bgcolor: '#fff', display: 'flex', alignItems: 'center',
               px: 2, borderBottom: '1px solid #EAECF0' }}>
      <IconButton><PanelLeft size={20} color="#004987" /></IconButton>
      <Box sx={{ width: '1px', height: 28, bgcolor: '#EAECF0', mx: 2 }} />
      <Typography sx={{ fontWeight: 600, fontSize: 18, flex: 1 }}>FSDH Payhub</Typography>
      <IconButton sx={{ bgcolor: '#F2F4F7' }}><Bell size={20} /></IconButton>
    </Box>
  );
}
```

In `DashboardLayout.tsx`, put the TopBar above the content and change the background:

```
<Box sx={{ display: 'flex', minHeight: '100vh', bgcolor: '#F3F9FF' }}>
  <Sidebar menuItems={menuItems} />
  <Box sx={{ flex: 1, ml: '280px' }}>
    <TopBar />
    <Box sx={{ p: 4 }}>{children}</Box>
  </Box>
</Box>
```

Step 6. Make fake data first

Build the screen with fake numbers so you are not waiting for the backend. Later you swap the fake file for a real server call, and the screen does not change.

```
// src/data/overviewMock.ts
export const collectionStats = {
  totalToday: 78_700_000_000, totalTodayChange: 9.8,
  thisMonth: 78_700_000_000,
  successful: 12800, successRate: 98.8,
  pending: 147, failed: 64,
};
export const payoutStats = {
  totalValue: 78_700_000_000, totalValueChange: 9.8,
  successful: 78_700_000_000, pending: 56, failed: 23,
};
export const volumeTrends = [
  { month: 'Jan', fsdh360: 62, fsdhPay: 40 }, { month: 'Feb', fsdh360: 63, fsdhPay: 41 },
  // ... one row per month up to Dec
];
export const inflowChannels = [
  { name: 'Static VA', value: 65 }, { name: 'Dynamic', value: 35 },
];
export const transactions = Array.from({ length: 7 }, (_, i) => ({
  id: `238423Acvs74${i}`, virtualAcct: '1000447873',
  collectionAcct: '1000345673', collectionAcctName: 'Revlant Int LTD',
  customer: 'Halima Kadiri', amount: 234_000_000_000,
  date: '2026-08-08T14:22:23', status: 'Successful', notification: 'Successful',
}));
```

Step 7. Helpers for money and dates

```
// src/utils/format.ts
export const nairaShort = (n: number) =>          // 78700000000 -> "₦78.7B"
  '₦' + new Intl.NumberFormat('en-NG', { notation: 'compact', maximumFractionDigits: 1
}).format(n);

export const nairaFull = (n: number) =>           // -> "₦234,000,000,000"
  '₦' + n.toLocaleString('en-NG');

export const longDate = (d = new Date()) =>       // -> "Friday, 23 July 2026"
  d.toLocaleDateString('en-GB', { weekday: 'long', day: 'numeric', month: 'long', year:
'numeric' });
```

Step 8. The StatCard component

One component, three looks. The `variant` decides the colours.

```tsx
// src/components/overview/StatCard.tsx
import { Box, Typography } from '@mui/material';
import { TrendingUp, type LucideIcon } from 'lucide-react';

type Variant = 'navy' | 'green' | 'plain';
interface Props {
  label: string; value: string; caption?: string; change?: number;
  icon: LucideIcon; iconColor?: string; valueColor?: string; variant?: Variant;
}

const bg = { navy: '#1D4E7E', green: '#067647', plain: '#fff' };

export default function StatCard({ label, value, caption, change, icon: Icon,
                                   iconColor, valueColor, variant = 'plain' }: Props) {
  const dark = variant !== 'plain';
  return (
    <Box sx={{ bgcolor: bg[variant], color: dark ? '#fff' : '#101828', borderRadius: '6px',
               p: 1.5, minHeight: 112, border: dark ? 'none' : '1px solid #EAECF0' }}>
      <Box sx={{ display: 'flex', justifyContent: 'space-between' }}>
        <Typography sx={{ fontSize: 12, opacity: dark ? 0.9 : 0.7 }}>{label}</Typography>
        <Icon size={20} color={dark ? '#fff' : iconColor} />
      </Box>
      <Typography sx={{ fontSize: 22, fontWeight: 700, mt: 1, color: valueColor }}>{value}
</Typography>
      {change !== undefined && (
        <Box sx={{ display: 'inline-flex', alignItems: 'center', gap: 0.5, mt: 0.5, px: 1,
                   borderRadius: 999, fontSize: 11, bgcolor: 'rgba(255,255,255,0.2)' }}>
          {change}% <TrendingUp size={12} />
        </Box>
      )}
      {caption && <Typography sx={{ fontSize: 10, opacity: 0.7, mt: 0.5 }}>{caption}
</Typography>}
    </Box>
  );
}
```

Using it for the collections row (5 cards in a row) and payouts row (4 cards):

```tsx
<Box sx={{ display: 'grid', gridTemplateColumns: 'repeat(5, 1fr)', gap: 2 }}>
  <StatCard variant="navy" label="Total collections today"
            value={nairaShort(c.totalToday)} change={c.totalTodayChange} icon={Wallet} />
  <StatCard label="Collections this month" value={nairaShort(c.thisMonth)}
            caption="August 2026 to Date" icon={WalletCards} />
  <StatCard label="Successful collections" value={`${c.successful.toLocaleString()}+`}
            caption={`${c.successRate}% Success Rate`} icon={CheckCircle2} iconColor="#12B76A"
/>
  <StatCard label="Pending collections" value={String(c.pending)}
            caption="Awaiting Action" icon={AlertCircle} iconColor="#F79009" />
  <StatCard label="Failed collections" value={String(c.failed)}
            caption="Requires Action" icon={XCircle} iconColor="#D92D20" />
</Box>
// Payouts row: same idea, gridTemplateColumns: 'repeat(4, 1fr)', first card variant="green",
// and the Failed payouts value uses valueColor="#D92D20".
```

For smaller screens use `gridTemplateColumns: { xs: '1fr 1fr', lg: 'repeat(5, 1fr)' }` so the cards wrap.

Step 9. The Volume Trends chart

```
// src/components/overview/VolumeTrendsChart.tsx
import { Box, Typography } from '@mui/material';
import { ResponsiveContainer, AreaChart, Area, XAxis, YAxis,
         CartesianGrid, Tooltip, Legend } from 'recharts';
import { volumeTrends } from '../../data/overviewMock';

export default function VolumeTrendsChart() {
  return (
    <Box sx={{ bgcolor: '#fff', border: '1px solid #EAECF0', borderRadius: '8px', p: 3 }}>
      <Typography sx={{ fontWeight: 600 }}>Volume Trends</Typography>
      <Typography sx={{ fontSize: 14, color: '#475467', mb: 2 }}>Daily Volume Trends in
Millions</Typography>
      <ResponsiveContainer width="100%" height={240}>
        <AreaChart data={volumeTrends}>
          <CartesianGrid vertical={false} stroke="#F2F4F7" />
          <XAxis dataKey="month" tickLine={false} axisLine={false} />
          <YAxis tickFormatter={(v) => `${v}m`} tickLine={false} axisLine={false} />
          <Tooltip />
          <Legend verticalAlign="top" align="right" iconType="circle" />
          <Area dataKey="fsdh360" name="FSDH 360" stroke="#0A2540" fill="#E4E9F0" />
          <Area dataKey="fsdhPay" name="FSDH Pay" stroke="#004987" fill="#F2F5F9" />
        </AreaChart>
      </ResponsiveContainer>
    </Box>
  );
}
```

Step 10. The Inflow Channel doughnut

```
// src/components/overview/InflowChannelChart.tsx
import { Box, Typography, Button } from '@mui/material';
import { ResponsiveContainer, PieChart, Pie, Cell } from 'recharts';
import { inflowChannels } from '../../data/overviewMock';

const COLORS = ['#007AFF', '#0023A2'];

export default function InflowChannelChart() {
  return (
    <Box sx={{ bgcolor: '#fff', border: '1px solid #EAECF0', borderRadius: '8px', p: 2 }}>
      <Typography sx={{ fontWeight: 600 }}>Inflow Channel</Typography>
      <Box sx={{ display: 'flex', alignItems: 'flex-start' }}>
        <ResponsiveContainer width="60%" height={220}>
          <PieChart>
            <Pie data={inflowChannels} dataKey="value" innerRadius="50%" outerRadius="100%"
                 startAngle={90} endAngle={-270} stroke="none">
              {inflowChannels.map((_, i) => <Cell key={i} fill={COLORS[i]} />)}
            </Pie>
          </PieChart>
        </ResponsiveContainer>
        <Box sx={{ mt: 2 }}>
          {inflowChannels.map((c, i) => (
            <Typography key={c.name} sx={{ fontSize: 13, display: 'flex', alignItems: 'center',
gap: 1 }}>
              <Box sx={{ width: 8, height: 8, borderRadius: '50%', bgcolor: COLORS[i] }} />
              {c.name} - {c.value}%
            </Typography>
          ))}
        </Box>
      </Box>
      <Box sx={{ borderTop: '1px solid #EAECF0', pt: 2, textAlign: 'right' }}>
        <Button variant="outlined" sx={{ borderRadius: 999, textTransform: 'none',
                color: '#101828', borderColor: '#D0D5DD' }}>View full report</Button>
      </Box>
    </Box>
  );
}
```

Put the two charts side by side: `display: 'grid', gridTemplateColumns: '2fr 1fr', gap: 3`.

Step 11. The transactions table

Use MUI's `Table`, `TableHead`, `TableBody`, `TableRow`, `TableCell`. Build it in this order:

1. **Tabs**: two buttons, "FSDH 360" and "FSDH PAY", kept in `useState`. The active one is `#003C70` with white text. Switching tab changes the title and which data is shown.
2. **Header row**: title "Collection Transactions (FSDH360)" with a small grey box showing the count (240), subtitle below. On the right: a search `TextField` with a `Search` icon at the start, and an outlined "Filters" button with the `ListFilter` icon.
3. **The table**: columns Transaction ID, Virtual Acct No, Collection Acct No (number on top, company name below in grey), Customer Name, Amount (bold, use `nairaFull`), Date/Time (date on top, time below), Status, Notification. The table is wider than the box, so wrap it in `<TableContainer sx={{ overflowX: 'auto' }}>`.

4. **Search**: keep the search text in `useState` and filter the rows: `rows.filter(r => JSON.stringify(r).toLowerCase().includes(search.toLowerCase()))`. Later the server should do this.
5. **Pagination**: "Page 1 of 10" on the left, "Previous" and "Next" outlined buttons on the right. Keep `page` in `useState`.

The status badge:

```
// src/components/overview/StatusPill.tsx
import { Box } from '@mui/material';

const styles = {
  Successful: { color: '#067647', bg: '#ECFDF3', border: '#ABEFC6' },
  Pending:    { color: '#B54708', bg: '#FFFAEB', border: '#FEDF89' },
  Failed:     { color: '#B42318', bg: '#FEF3F2', border: '#FECDCA' },
};

export default function StatusPill({ status }: { status: keyof typeof styles }) {
  const s = styles[status];
  return (
    <Box sx={{ display: 'inline-flex', alignItems: 'center', gap: 0.75, px: 1, py: 0.25,
               borderRadius: 999, fontSize: 12, fontWeight: 500,
               color: s.color, bgcolor: s.bg, border: `1px solid ${s.border}` }}>
      <Box sx={{ width: 6, height: 6, borderRadius: '50%', bgcolor: s.color }} />
      {status}
    </Box>
  );
}
```

Step 12. Put it all together on the page

```
// src/pages/overview/Overview.tsx
import { Box, Typography, Button } from '@mui/material';
import DashboardLayout from '../../components/DashboardLayout';
import { payhubNavItems } from '../../config/navigation';
import { longDate } from '../../utils/format';
import CollectionsRow from '../../components/overview/CollectionsRow';
import PayoutsRow from '../../components/overview/PayoutsRow';
import VolumeTrendsChart from '../../components/overview/VolumeTrendsChart';
import InflowChannelChart from '../../components/overview/InflowChannelChart';
import TransactionsTable from '../../components/overview/TransactionsTable';

export default function Overview() {
  return (
    <DashboardLayout menuItems={payhubNavItems}>
      <Box sx={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', mb: 3
}}>
        <Box>
          <Typography sx={{ fontWeight: 600, fontSize: 18 }}>OVERVIEW</Typography>
          <Typography sx={{ color: '#475467' }}>
            Consolidated position across FSDH 360 collections and FSDH Pay-outs -{' '}
            <b>{longDate()}</b>
          </Typography>
        </Box>
        <Button variant="contained" sx={{ bgcolor: '#003C70', textTransform: 'none',
                borderRadius: '4px', px: 2.5, py: 1.25 }}>Generate Report</Button>
      </Box>

      <Typography sx={{ fontSize: 12, fontWeight: 700, mb: 1 }}>COLLECTIONS - FSDH
360</Typography>
      <CollectionsRow />
      <Typography sx={{ fontSize: 12, fontWeight: 700, mt: 3, mb: 1 }}>PAYOUTS - FSDH
PAY</Typography>
      <PayoutsRow />

      <Box sx={{ display: 'grid', gridTemplateColumns: { xs: '1fr', lg: '2fr 1fr' }, gap: 3, mt:
3 }}>
        <VolumeTrendsChart />
        <InflowChannelChart />
      </Box>

      <Box sx={{ mt: 3 }}><TransactionsTable /></Box>
    </DashboardLayout>
  );
}
```

Then in `App.tsx` point `/dashboard` at it:

```
import Overview from './pages/overview/Overview';
...
<Route path="/dashboard" element={
  <ProtectedRoute allowedTypes={['fulltime']}><Overview /></ProtectedRoute>
} />
```

Step 13. Swap fake data for real data (later)

1. Ask the backend team for the addresses that return the overview numbers, the chart data and the transactions list.
2. Create `src/services/api/overviewService.ts` in the same style as `employeesService.ts` (one small function per address).
3. In each component, replace the import from `overviewMock` with a call to the service inside `useEffect`, store the result in `useState`, and show an MUI `Skeleton` while it loads.

9. Your folder structure when you are done

```
src/
  App.tsx                          routes: "/" -> SignIn, "/dashboard" -> Overview
  index.tsx                        uses createRoot
  index.css
  assets/
    fonts/                         Avenir files, if you get them
    images/
      fsdh-merchant-logo.svg
      auth-bg-pattern.png
      slide-1.svg  slide-2.svg  slide-3.svg
  components/
    AuthBootstrap.tsx
    ProtectedRoute.tsx
    DashboardLayout.tsx            now includes TopBar
    Sidebar.tsx                    restyled, grouped sections
    TopBar.tsx                     NEW
    auth/
      AuthCarousel.tsx             NEW
    overview/
      StatCard.tsx                 NEW
      CollectionsRow.tsx           NEW
      PayoutsRow.tsx               NEW
      VolumeTrendsChart.tsx        NEW
      InflowChannelChart.tsx       NEW
      TransactionsTable.tsx        NEW
      StatusPill.tsx               NEW
  config/
    colors.ts                      NEW
    navigation.ts                  payhubNavItems with sections
    sessionKeys.ts
  data/
    overviewMock.ts                NEW (fake data until the API is ready)
  pages/
    NotFound.tsx
    auth/
      SignIn.tsx                   NEW (replaces LandingPage)
      SignInForm.tsx               NEW
      SessionLanding.tsx
    overview/
      Overview.tsx                 NEW (replaces Dashboard)
  services/
    api/
      httpClient.ts
      index.ts
      authService.ts               NEW
      overviewService.ts           NEW (later)
      employeesService.ts
    auth/
      signIn.ts                    NEW
  store/
    index.ts  authSlice.ts  onboardingSlice.ts
  utils/
    format.ts                      NEW
```

10. Order of work and a final checklist

Work in this order. Commit to Git after each numbered item so you can always go back.

1. Do the tidy-ups in section 5 (index.tsx, index.css, title). Run `npm run dev` and check nothing broke.
2. Sort out fonts (get Avenir or install Nunito Sans).
3. Export the images from Figma (section 7, step 1).
4. Build the Sign In layout and form with a mock login. Check: empty fields show errors, a bad email shows an error, Sign in takes you to `/dashboard`.
5. Build the carousel. Check: arrows and dots work, it moves on by itself.
6. Run `npm install recharts`.
7. Add `colors.ts`, `format.ts` and `overviewMock.ts`.
8. Update `navigation.ts`, restyle the Sidebar, add the TopBar.
9. Build `StatCard` and the two rows of cards.
10. Build the two charts.
11. Build `StatusPill` and the transactions table with tabs, search and paging.
12. Put it all together in `Overview.tsx` and point `/dashboard` at it.
13. Hold the browser next to the Figma image and compare spacing, sizes and colours. Fix what differs.
14. Run `npm run lint` and `npm run build`. Both must finish without errors.
15. When the backend is ready, swap the mock login and mock data for real calls.

> **Questions to ask the team**
>
> - Backend: the login address and its reply; the addresses for overview stats, chart data and transactions; the real user roles.
> - Designer: the Avenir font files; the text for carousel slides 1 and 3; the exported logo, background and slide images; designs for the Reset Password and Sign Up screens (the links have nowhere to go yet).

11. Small dictionary

Word	Meaning
Component	A reusable piece of screen, written as a function that returns what to show. Example: `Sidebar`.
Props	The settings you pass into a component, like `<StatCard label="Pending" />`.
State (`useState`)	Something a component remembers that can change, like which tab is selected.

`useEffect`	Code that runs after the screen appears, for example to load data from the server.
Route	A rule that says "for this web address, show this page".
Redux store / slice	The app-wide memory. A slice is one section of it (here: `auth` and `onboarding`).
Dispatch / action	Sending a message to the store to change it, like `dispatch(logout())`.
Selector	Reading from the store, like `useAppSelector(s => s.auth.user)`.
localStorage	A small storage box inside the browser that survives a page refresh.
Token	A long secret string the server gives after login. It is sent with every request to prove who you are.
401	The server's way of saying "you are not logged in, or your login has expired".
`sx`	MUI's way of writing styles directly on a component.
Mock	Fake data or a fake login used while the real one is not ready.